

Name of the Student	MADDIRALA BALAMURALI KRISHNA
Reg. No	192325015
GitHub Link	https://github.com/krishnalsk/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE:

MLA0403

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

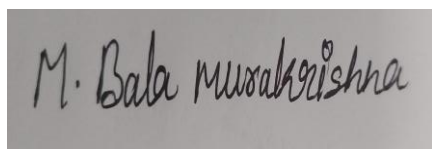
Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I MADDIRALA BALAMURALI KRISHNA (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



M.BALA MURALI KRISHNA

Signature of the student:

Name of the student:

Q1. Unsupervised Training of Neural Networks — Autoencoders

Scenario: A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Recommended Technique: Autoencoder-based Anomaly Detection

Since the ECG records are unlabeled, supervised classification cannot be used directly. An Autoencoder, trained through unsupervised learning, is the most suitable technique because it learns to reconstruct "normal" patterns and flags any pattern that it cannot reconstruct well as abnormal.

Justification

- No labels are required — the autoencoder only needs a large set of ECG signals that are predominantly normal.
- Autoencoders learn a compressed latent representation that captures the essential shape of a normal heartbeat (P-QRS-T wave).
- Abnormal beats deviate from the learned normal representation, producing a measurably higher reconstruction error, which acts as an anomaly score.
- The approach scales well to thousands of records since training is unsupervised and can run in batches/mini-batches.

Proposed Workflow

- 1. Data Preparation: Segment continuous ECG signals into fixed-length beats/windows; normalize amplitude and remove baseline noise.
- 2. Model Design: Build an encoder (series of dense or 1D-convolutional layers) that compresses each ECG window into a low-dimensional latent vector, followed by a symmetric decoder that reconstructs the original signal.
- 3. Training: Train the autoencoder using only (or mostly) normal ECG beats, minimizing reconstruction loss (e.g., Mean Squared Error) using an optimizer such as Adam.
- 4. Threshold Selection: After training, compute reconstruction error on a validation set of normal beats and set an anomaly threshold (e.g., mean + 3×standard deviation).
- 5. Detection: For new/unseen ECG beats, pass them through the trained autoencoder; if reconstruction error exceeds the threshold, flag the beat as a potential abnormal pattern for clinician review.
- 6. Validation: Periodically validate flagged cases against clinician-confirmed diagnoses to refine the threshold and retrain the model as more data becomes available.

Q2. Autoencoders — Representation Learning for Dimensionality Reduction

Scenario: A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

How an Autoencoder Helps

An autoencoder is a neural network trained to reproduce its own input at the output layer, while passing the data through a narrow "bottleneck" hidden layer. This bottleneck forces the network to learn a compact, information-rich representation of the data — effectively performing non-linear dimensionality reduction, similar in spirit to PCA but capable of capturing non-linear relationships that PCA cannot.

Application to Customer Segmentation

- **Input Features:** Represent each customer as a high-dimensional vector — purchase frequency, spend per category, recency, basket size, discount usage, etc.
- **Encoder:** Compresses this high-dimensional vector into a small latent vector (e.g., from 50+ features down to 5–10 dimensions) that preserves the key purchasing behavior patterns.
- **Decoder:** Reconstructs the original feature vector from the latent representation; training minimizes reconstruction error so the latent code retains maximum useful information.
- **Segmentation Step:** Once trained, discard the decoder and use only the encoder's latent vectors as compact customer representations.
- **Clustering:** Apply a clustering algorithm (e.g., K-Means, DBSCAN, or hierarchical clustering) on the low-dimensional latent vectors to form customer segments — this is far more effective and computationally efficient than clustering on the original noisy, high-dimensional data.

Benefits

- Removes redundancy and noise in the original purchase data.
- Captures non-linear relationships between purchasing attributes.
- Produces well-separated, visualizable clusters (e.g., via 2D/3D latent projections) that make marketing segmentation more interpretable and actionable.

Q3. Width and Depth of Neural Networks, Activation Functions

Scenario: A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Problem with a Shallow Network

A shallow network has limited capacity to learn the complex, hierarchical features required for facial recognition (edges → facial parts → identity-specific features). It underfits the data and cannot capture the fine-grained variations needed to distinguish faces.

Recommended Architectural Changes

- 1. Increase Network Depth:** Use a deep Convolutional Neural Network (CNN) with multiple convolutional and pooling layers stacked hierarchically. Depth allows the network to learn a feature hierarchy — early layers detect edges and textures, middle layers detect facial components (eyes, nose, mouth), and deeper layers learn identity-discriminative embeddings.
- 2. Increase Network Width (where appropriate):** Increase the number of filters/channels per convolutional layer so the network can capture a richer variety of features at each level, improving representational capacity without excessive depth-related issues.
- 3. Use Residual Connections:** Adopt an architecture such as ResNet-style skip connections so that increasing depth does not lead to vanishing gradients or degradation in training accuracy.
- 4. Replace/Upgrade Activation Functions:** Replace Sigmoid or Tanh activations (which saturate and cause vanishing gradients in deep networks) with ReLU or its variants (Leaky ReLU, ELU, or Swish) in hidden layers, since these maintain healthy gradient flow through many layers.
- 5. Output Layer Activation:** Use Softmax at the output layer for identity classification, or an embedding layer with a triplet/contrastive loss (common in modern face recognition systems such as FaceNet) for verification-style tasks.
- 6. Regularization:** Add Batch Normalization after convolutional layers and Dropout in fully connected layers to stabilize training and reduce overfitting as depth increases.

Expected Outcome

These changes allow the network to learn increasingly abstract and discriminative facial features layer by layer, substantially improving recognition accuracy compared to the original shallow architecture.

Q4. Restricted Boltzmann Machines (RBMs) — Unsupervised Recommendation

Scenario: A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Why RBMs Fit This Problem

A Restricted Boltzmann Machine is a two-layer, undirected, generative neural network (visible layer + hidden layer, with no intra-layer connections) that learns a probability distribution over its inputs in a purely unsupervised manner — making it well suited to learning patterns from viewing history without needing labels.

RBM-based Recommendation Workflow

- 1. Data Representation: Represent each user as a visible-layer vector where each unit corresponds to a movie, with a binary value (watched/not watched) or a normalized rating/engagement score.
- 2. Model Structure: The visible layer connects fully to a hidden layer of latent "taste" units; there are no visible-visible or hidden-hidden connections (restricted structure), which keeps inference tractable.
- 3. Training (Contrastive Divergence): Train the RBM using Contrastive Divergence — the network learns weights that make the hidden units capture latent genre/taste factors that explain co-viewing patterns across users.

- 4. Reconstruction: For a given user, feed their known viewing vector forward to activate hidden units (learned taste factors), then reconstruct the visible layer backward — the reconstructed probabilities for movies the user has NOT yet watched serve as predicted preference scores.
- 5. Recommendation Generation: Rank the unwatched movies by their reconstructed activation probability and recommend the top-N highest-probability titles.
- 6. Scaling: Stack RBMs into a Deep Belief Network for richer, hierarchical taste representations if a single RBM's capacity is insufficient.

Advantages

- Works purely from implicit viewing history — no explicit labels or ratings required.
- Naturally handles missing data (unwatched movies) through the generative reconstruction process.
- Learns latent taste/genre factors automatically, similar in spirit to matrix factorization but with non-linear representational power.

Q5. Deep Learning for Defect Detection — Representation Learning

Scenario: A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

Problem Characteristics

The data is high-frequency, sequential sensor data (time-series), defects are likely rare compared to normal operation, and labeled defect examples may be scarce — so the solution must combine representation learning with a suitable architecture for sequential data.

Proposed Deep Learning Solution

- 1. Data Preprocessing:** Aggregate multi-sensor readings into fixed-length time windows; normalize/scale each sensor channel; handle missing values via interpolation.
- 2. Representation Learning Stage:** Train an autoencoder (or a 1D-CNN/LSTM autoencoder) on sensor windows from normal operating conditions to learn a compact latent representation of "healthy" machine behavior, since defective examples are rare and largely unlabeled.
- 3. Architecture Choice:** Use 1D Convolutional layers to extract local temporal patterns (spikes, drifts) from each sensor stream, followed by LSTM/GRU recurrent layers to capture longer-term temporal dependencies across the second-by-second readings.
- 4. Anomaly / Defect Scoring:** As in Q1, compute reconstruction error between the input window and the autoencoder's output; a high error indicates a deviation from normal behavior, i.e., a probable defect.
- 5. Fine-tuning with Limited Labels:** If a small set of confirmed defective examples is available, fine-tune a classifier head (dense + Softmax/Sigmoid) on top of the learned latent representations — this is a semi-supervised/transfer-learning approach that makes efficient use of scarce labels.
- 6. Real-time Deployment:** Deploy the trained model in a streaming pipeline that scores each incoming one-second window in real time, triggering an alert when the anomaly score/defect probability crosses a calibrated threshold.

